# 8/31 8회차 스터디



▼ 예습 해오기!

다음 8주차 스터디 공지입니다!

코멘토 자료는 1주차 내용 미리 읽어와 주시면 감사하겠습니다.

유튜브 영상은 아래 구간만 봐도 이번에 다룰 AWS 서비스들의 큰 흐름을 잡는 데 도움이 됩니다.
https://www.youtube.com/watch?v=iX6X0rNmgzY&t=908s
( 3:28 ~ 15:06)

그리고 노션 8주차의 「서버 스터디한 내용과 연계되는 AWS 서비스들 정리」 토글도 한 번 읽어와 주세요.
지금까지 사용한 Docker, Nginx, PostgreSQL, 로드밸런싱, 모니터링 구조가 AWS에서는 어떤 서비스와 연결되는지 보는 내용입니다.

▼ 서버 스터디한 내용과 연계되는 AWS 서비스들 정리

| PDF의 AWS 서비스·리소스 | 자료 주차 | PDF에서 하는 일 | 우리가 만든 것과의 연결 |
|---|---|---|---|
| AWS 계정·IAM | 1주차 | AWS 콘솔·리소스를 사용할 사용자와 권한 관리 | 로컬 Docker에는 없는 클라우드 고유 영역. “누가 AWS 서버를 만들고 수정할 수 있는가”를 정함 |

| PDF의 AWS 서비스·리소스 | 자료 주차 | PDF에서 하는 일 | 우리가 만든 것과의 연결 |
|---|---|---|---|
| Amazon VPC | 1·3주차 | AWS 안에서 서버들을 둘 사설 네트워크 공간 생성 | Docker Compose 네트워크(`proxy-net`, `db-net`)를 더 큰 클라우드 단위로 확장한 개념 |
| Subnet | 1·3주차 | VPC를 공개 구역·비공개 구역으로 나눔 | Nginx만 공개하고 app·PostgreSQL은 내부에 둔 구조를 AWS 네트워크 구역으로 표현 |
| Route Table·Internet Gateway | 1주차 | 어떤 Subnet이 인터넷과 통신할 수 있는지 결정 | `ports: "8080:80"`으로 Nginx만 외부에서 받게 한 것과 같은 목적: 외부 요청의 입구를 정함 |
| Security Group | 1·3주차 | 서버별 허용 포트·접근 대상을 설정 | Nginx 80/8080 공개, app은 Nginx만 접근, DB는 app만 접근하도록 구성한 규칙에 대응 |
| EC2 | 1·3주차 | 클라우드에서 앱이나 웹 서버를 실행할 가상 컴퓨터 | 현재는 노트북에서 Docker가 실행되지만, AWS에서는 EC2가 그 실행 환경이 될 수 있음 |
| EBS | 1주차 | EC2에 연결하는 영구 디스크 | Docker Named Volume과 비슷하게 데이터가 컨테이너/서버 종료 뒤에도 남게 하는 저장 공간 |
| AMI | 1주차 | EC2 서버 설정을 복제하기 위한 이미지 | Docker 이미지와 비슷하게 "실행 환경을 재사용"하지만, AMI는 가상 컴퓨터 전체 이미지 |
| Amazon RDS | 3·4주차 | AWS가 관리하는 MySQL DB 생성·연결·관찰 | PostgreSQL 컨테이너와 Named Volume을 직접 관리했던 것을 관리형 DB로 바꾼 형태 |
| DB Subnet Group | 3주차 | RDS DB를 비공개 Subnet들에 배치 | PostgreSQL을 Nginx와 직접 연결하지 않고 `db-net` 안쪽에 둔 구조와 연결 |
| Application Load Balancer | 3주차 | 외부 요청을 여러 EC2에 분산 | Nginx가 앞에서 `app1`, `app2`에 요청을 분산한 역할과 가장 직접적으로 대응 |

| PDF의 AWS 서비스·리소스 | 자료 주차 | PDF에서 하는 일 | 우리가 만든 것과의 연결 |
|---|---|---|---|
| Target Group | 3주차 | ALB가 요청을 전달할 앱 서버 목록 | Nginx의 `upstream petclinic_backend { app1:8080; app2:8080; }` 에 대응 |
| ALB Health Check | 3주차 | 정상 앱 서버만 요청 대상으로 유지 | app 하나를 중지했을 때 Nginx가 다른 app으로 우회한 실습과 대응 |
| Amazon CloudWatch | 4주차 | RDS·EC2 등 AWS 리소스의 상태·성능 지표 확인 | Prometheus·Grafana가 PetClinic/JVM 중심이었다면, CloudWatch는 EC2·RDS·ALB 같은 AWS 자원 중심 |
| AWS Lambda | 4주차 | 서버를 직접 계속 실행하지 않고, 요청·이벤트 때 코드 실행 | 현재 PetClinic/Nginx 구조의 대응물이 아님. 항상 실행되는 서버 방식과 비교하기 위한 별도 서버리스 방식 |
| Amazon API Gateway | 4주차 | Lambda 같은 API 앞에서 HTTP 요청을 받는 입구 | Lambda 실습과 한 묶음. Nginx/ALB와 비슷하게 요청 입구 역할을 하지만 서버리스 API에 주로 사용 |
| CloudFront | 4주차 과제 | 사용자 가까이에서 정적 파일을 캐시·제공 | 4주차 Nginx 정적 파일 캐시와 연결되지만, 인터넷 전 세계 사용자 앞단에 두는 CDN이라는 차이 |
| JMeter | 4주차 | DB·웹 요청 부하 생성 | AWS 서비스가 아니라 부하 테스트 도구. 우리가 사용한 k6와 같은 범주의 도구 |

정리하면, 이번 8주차에서 지금까지의 실습과 **직접 이어서 집중할 대상**은 아래입니다.

```
Docker 네트워크
→ VPC · Subnet · Route Table · Security Group

노트북에서 Docker 실행
→ EC2

PostgreSQL 컨테이너 + Named Volume
→ RDS

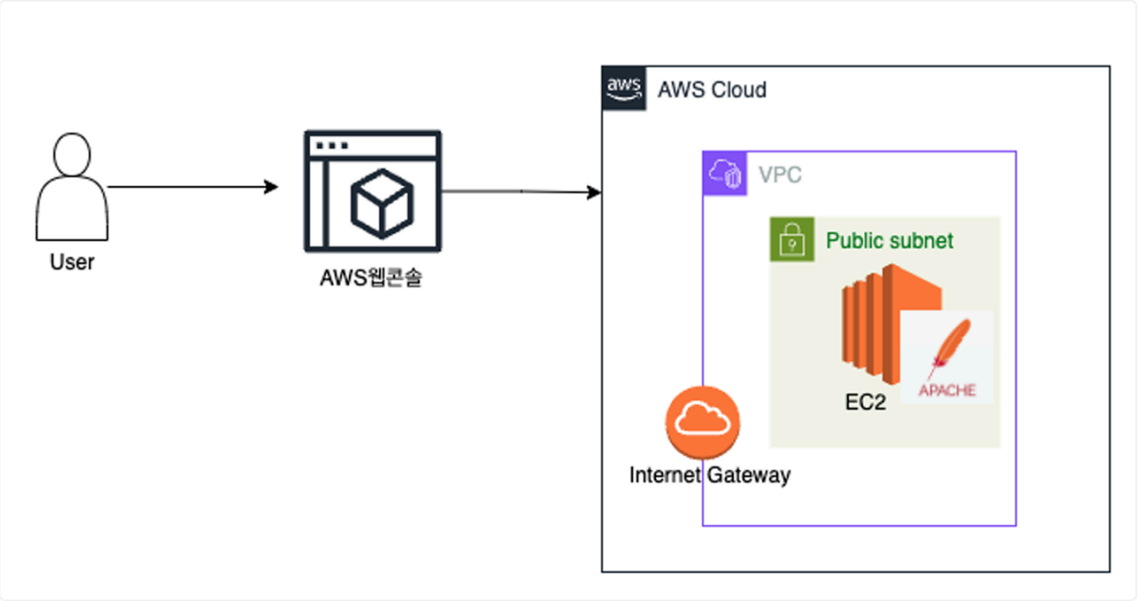
Nginx + app1/app2 + 장애 우회
→ ALB + Target Group + Health Check
```

```
Prometheus · Grafana
→ CloudWatch와 역할 비교
```

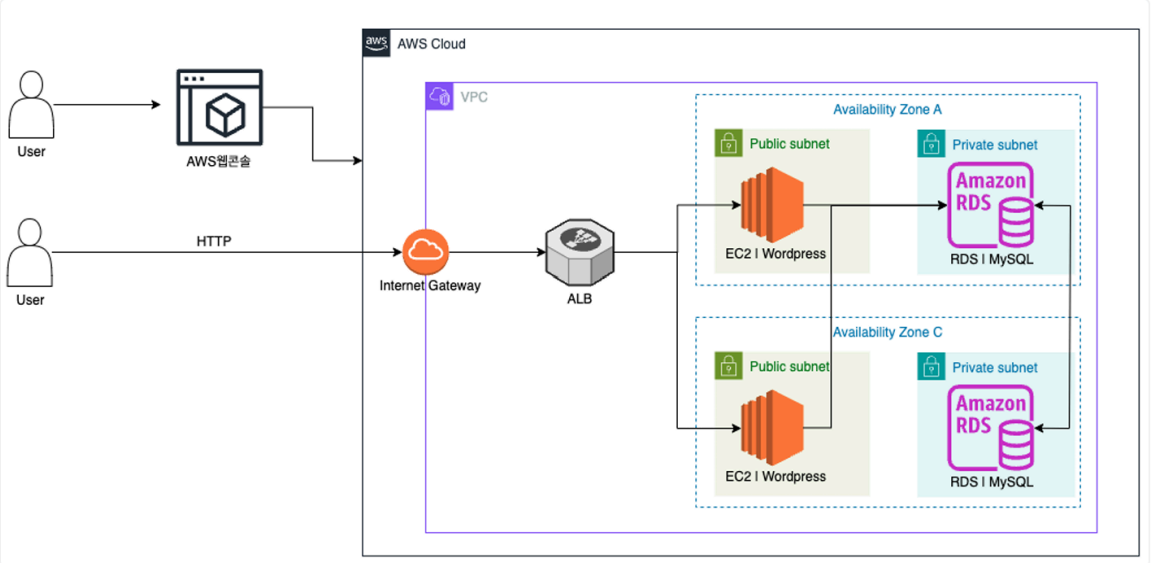
# 만들어 볼 것들

https://comento.kr/edu/learn/ITSW/IT-G956

▼ 1주차

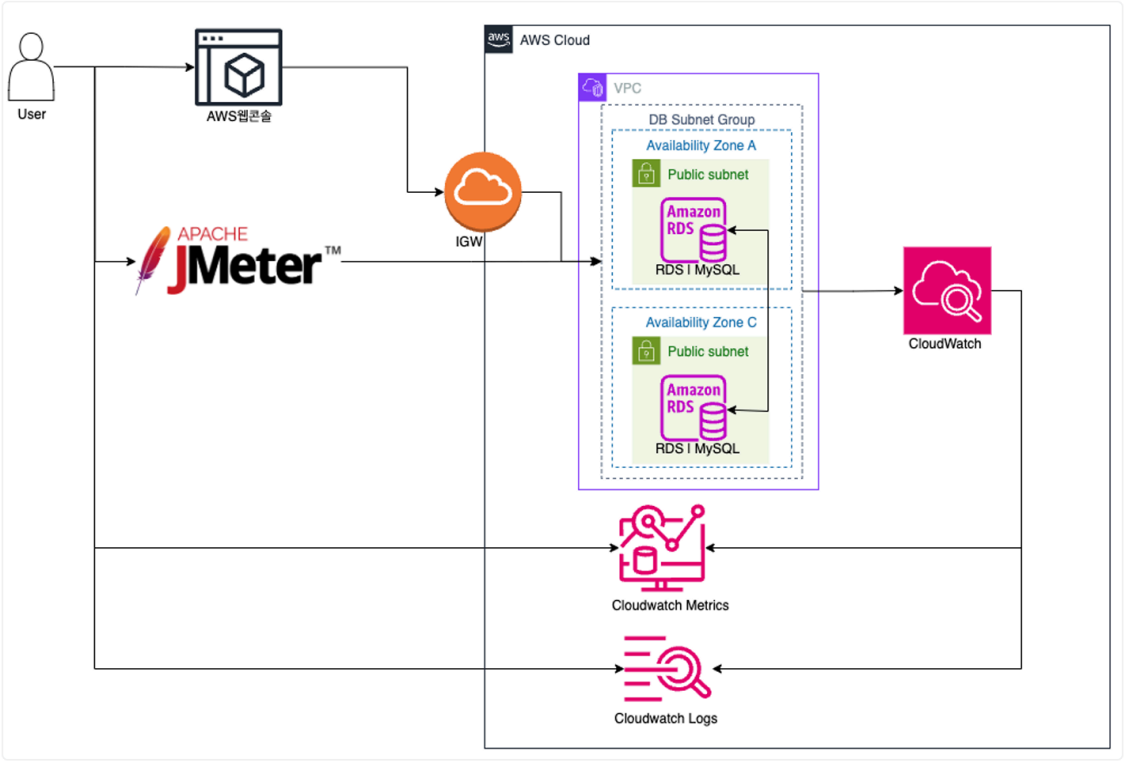


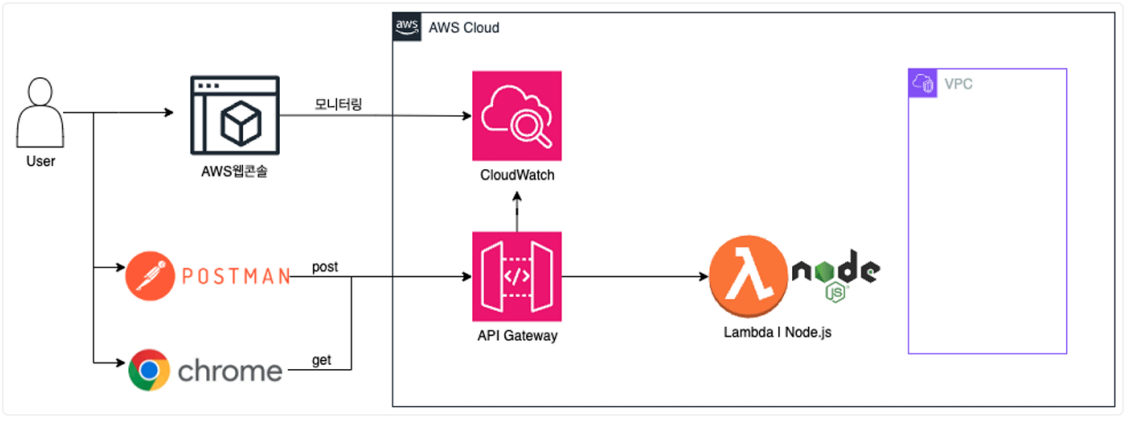

▼ 3주차



| PetClinic 실습 | 3주차 WordPress 실습 |
| --- | --- |
| Nginx | Apache |
| PetClinic | WordPress |
| 내장 Tomcat | PHP 실행 환경 |
| PostgreSQL 컨테이너 | RDS MySQL |
| Nginx upstream app1·app2 | ALB Target Group의 EC2 여러 대 |

▼ 4주차



```
sudo dnf install mariadb105 -y
```